

第3章

ガードレールによる境界管理と外部ツール／サービスの活用

AIコーディングに正しさを求める試み

「同じ指示をしたとしても、同じ答えが返ってくるとは限らない」というのは、今ではよく知られた生成AIの性質です。しかし、Vibe Codingで質の高いコードを生成しようと思うのであれば、この揺らぎと向き合う必要があります。本章では、Vibe Codingの「ガードレール」を作成する最新の手法を解説します。

Author ryoppiippi GitHub ryoppiippi

Vibe Codingにおける「ガードレール」とは



Vibe Coding、すなわち Agent Coding において、生成されるコードの品質を担保するためには、大きく「前処理」「処理中」「処理後」の3つの視点が必要です。これらの用語は、機械学習における前処理、モデルそのものの処理、後処理、に対応しています。Vibe Coding においてもこの3つに重点を当ててコードの品質を保っていくというのが、本章で伝えたいアイデアです。質の高いコードを生成するためには、それぞれの過程でたとえば次のような視点が求められます。

- ・ どうドキュメントを渡すか (前処理)
- ・ 実装しているものが仕様に沿っているか、やるべきことをきっちりこなしているか (処理中)
- ・ コードが正しく動くか、要件を満たしているか (後処理)

これらの概念は、人間がコードを書く場合においても必要なものとして議論されてきた考え方とオーバーラップしています。

本章では、後処理、すなわちコードが正しく動作するかという点に特に重点を置いて議論を進めます。AIの生成するコードは非決定的 (Non-deterministic) な性質を持ちます。同じ

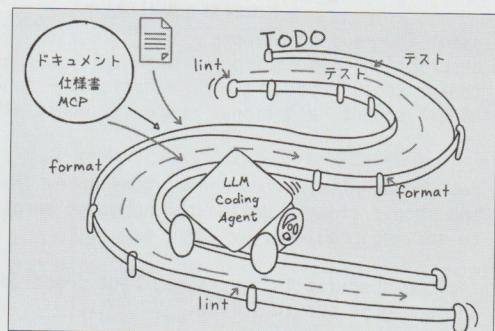
プロンプトを与えても、毎回同じコードが生成されるとは限りません。LLMの知識に基づいたコードは出力されますが、出力には本質的に「揺らぎ」が生じます。書きっぱなしのAI生成コードが本当に正しいかどうか、決定的 (Deterministic) に検証する手段がなければ、安心してデプロイすることはできません。

Vibe Coding における「ガードレール」とは、この「コードを決定的に検証する手段」のことを指します。コードが正しく動作するかを検証するにあたって、ガードレールは非常に重要なファクターです (図1)。

本章ではガードレールとして、大きく次の3つのことについてお話します。

- ・ ライブラリやフレームワークの仕様を LLM に正確に伝える方法 (前処理)

▼図1 ガードレールのイメージ



- ・仕様に沿っているかを自動的に検証する方法（処理中）
- ・生成されたコードが正しく動くかを自動的に検証する方法（後処理）

まずは、ライブラリやフレームワークの情報を正確に伝えるために、どのような方法があり、現時点でどんな方法が最適なのかを解説していきます。

LLMに正しい情報を与える必要性



Agent Codingで使用するLLMには「カットオフ」という概念が存在します。カットオフ（Knowledge Cutoff Date）とは、そのモデルの事前学習データが収集された最終時点を表す概念です^{注1}。カットオフに含まれているデータはモデルに学習されているため望む出力が得られますが、カットオフ以降に発生した出来事や公開された情報について、LLMは本質的に知識を持ちません。

カットオフはたいいていの場合、モデルが公開されたタイミングの数ヵ月以上前、場合によっては半年以上前に設定されています。半年という期間は、IT業界にとっては非常に長い時間であり、その間に公開された新しいパッケージのバージョンには当然対応していません。

古い情報に基づいて生成されたコードは、正しく動作しなかったり、品質が低くなったりする可能性があります。たとえば、最近のモデルでは問題にならなくなりましたが、2025年4月ごろにおいては、React Router v7のコードを出力する際に、LLMがv6以前のコードを生成してしまう問題が報告されていました。Svelteについても同様の問題が報告されており、Svelte 5のコードを出力させようとしても、

Svelte 4以前の古い構文が生成されてしまうケースがありました^{注2}。

さらに、知らない知識について間違った情報を返してしまうハルシネーションも課題として存在します。メジャーな言語と比べて情報量が少ない比較的マイナーな言語、たとえばZig、Gleam、MoonBitなどを生成しようとする、的的外れな出力が生成されてしまい、期待したものが出てこないことがあります。

そのため、品質の高いコードを生成するためには、モデル自体が持つ知識に加えて、なんらかの手段で、利用するプログラミング言語やフレームワークについての最新かつ正しい情報をLLMに伝えておく必要があります。言語やフレームワークだけでなく、ライブラリに対する知識もアップデートさせる必要があります。執筆現在（2025年12月）において登場している最新のモデルでは、こういった問題は改善しつつあるものの、正確な情報を提供することは依然として重要です。

知識をLLMに渡す手段の変化



LLMへ情報を渡す手段としては、技術の進化や人々のAIの扱い方の変化に応じて、さまざまなものが生まれました。まずはじめに、ここ数年でいかにLLMへの情報を渡す手段が変化してきたかを説明します。

RAG

Clineなどのコーディングエージェントが登場する前からある、LLMに情報を渡す方法としては、RAG（Retrieval-Augmented Generation）が存在します。これは事前に保持しているある文章やデータベース（ナレッジとも言います）などから、プロンプトに応じてLLMに関連する情報を一緒に渡すことで、LLMの回答の精度を

注1) Cheng et al., "Dated Data: Tracing Knowledge Cutoffs in Large Language Models", arXiv: 2403.12958, 2024: <https://doi.org/10.48550/arXiv.2403.12958>

注2) <https://scottspence.com/posts/ai-coding-tools-actually-use>

Vibe Coding完全攻略

コードを1行も書かない、新時代のAIプログラミング手法

上げるというものです。当時はドキュメントをベクトル表現にし、プロンプトと近い情報を自動的に取ってきて、一緒にLLMに渡すことで回答の精度を上げるというしくみをしていました。しかし、RAGは繰り返し処理、ループが必要なコーディングエージェントにおいてはそれほど使われることはありませんでした。

Web 検索

次に出てきたのがWeb検索です。プロンプトを介して、Googleを検索するようにAIがWebの検索を勝手に行って情報を集め、それを基に回答を生成するというしくみです。AIにWeb検索を実行させる試みはChatGPTから始まり、各社のLLMサービスに組み込まれました。Webの検索を人間がやらなくてよく、またWebの情報を使えるという点で有用な反面、毎回ひとつひとつ情報を取ってこないといけないため速度が遅いのと、確実にほしい情報を得られる保証がなく、ある程度は博打^{ばくち}であることがデメリットとして挙げられます。

現在の各社のコーディングエージェントにも搭載されている機能ですが、これに頼るのは最終手段だと思ったほうがよいでしょう。

llms.txt

次に登場したのがllms.txtです^{注3}。llms.txtはWebサイトのルートに置かれるテキストファイルで、そのWebサイトに関するドキュメントの場所を記述するためのしくみです。フレームワークやライブラリなどの公式から情報を提供する手法として、執筆時点でもある程度普及しています。

従来のWebサーチやWebフェッチなどでWebサイトのデータを取得する場合、HTML形式で情報を取得することになります。HTMLというものはUIの情報を含むものであり、純正な情報を取る手段としては必要のない情報が

付随してしまっているため、トークンあたりの情報量としては効率が良くありません。

llms.txtはMarkdown形式で情報が記述されており、トークンを最小限に抑えたAIフレンドリーなデータになっています。また、RAGではドキュメントを各々が保管していたのに対して、llms.txtではライブラリやサービス公式がネットワーク越しにドキュメントを提供するため、正確性が高いことが特徴です。たとえば、Svelte、Bun、Cloudflare Developersなどがllms.txtを提供しています。また、Mintlify^{注4}のようなドキュメント生成サービスは、Markdownテキストを提供するだけでなく、コーディングエージェントからのアクセスがあった場合に自動的にHTMLではなくMarkdownを返すしくみを持っているものもあります。

このころは、コーディングエージェントにllms.txtのURLを渡し、curlなどのCLIで情報を取得させていました。

ツール呼び出しとMCP

2024年末から今年の頭にかけて爆発的に普及したのが、MCPと呼ばれるプロトコルです。MCPが登場した背景を説明するにあたって、まずLLMにおける「ツール呼び出し (tool calling)」という概念について触れておきます。

LLMによるツール呼び出しは、2023年6月にOpenAIから発表され、その後急速に普及した概念です。LLMそのものだけでは学習された知識のみでの受け答えになるので、そもそも新しい情報を取得することもできませんし、また外部に何か作用することもできません。そのため、LLMはツール呼び出しによって、外部とやりとりを行います。

ツール呼び出しは、必要に応じてLLMが、外部とやりとりをするための手段 (ツール) を呼び出すかどうかを判断し、実際に適切にやり

注3) <https://llmstxt.org/>

注4) <https://www.mintlify.com/blog/simplifying-docs-with-llms-txt>

とりをすることができるというしくみです。これを活用することで、LLMの能力を限りなく引き上げることができます。

しかし、ツール呼び出しのしくみが登場した2023年時点では、ツール呼び出しの標準的な規格がなく、各社のLLMがそれぞれ異なった方法によってサポートしていました。各社が独自に提供していたツール呼び出しのしくみですが、それらの実装はいずれもJSONスキーマを基にして策定されていることから、どのツールもどのLLMともシームレスに連携できるよう共通化しよう、という機運が高まりました。その結果として策定されたのがMCPです。

MCP (Model Context Protocol) は、LLMがツールを呼び出すときのプロトコルを標準化した規格です^{注5}。MCPは基本的に外部、つまりLLMの外の世界とやりとりするためのプロトコルです。もちろん何か作用をすることにも使えますし、今までのとおり情報を取ってくることもできます。

MCPの優れている点は、ツールを呼び出す際にツールの説明文とタイトルの設定が必須となっていること、そしてパラメータに説明文が必須となっていることです。この設計は、LLMがツールを呼び出すときの成功率向上に寄与しています。

MCPには大きく分けて2つの種類があります。

- ・リモートMCP：REST APIのようにエンドポイントとして提供される。LLMはHTTPを通じて外部サービスと接続、連携できる
- ・ローカルMCP：ローカルのCLIとLLMをつなぐ。CLIツールだけでは不十分な場合に、MCPを介することでLLMとのインタラクションが可能になる

LLMが何かドキュメントを必要としたときに、MCPとして登録されているツールを呼び

出すことで、効果的に情報を取得することができます。現時点でも、さまざまなライブラリ、ツール、サービスのMCPが公開されています。DeepWikiやContext7、grep.appといった、各社ライブラリが公式で提供しているMCPライブラリやサーバも数多くありますし、汎用的に情報を検索してくるMCPもあります。また、FigmaやGitHub、Jira、Linearは、それぞれが提供するサービスに特化したMCPを提供しており、LLMはこのMCPを介して、たとえばサービスが持っている社内特定のコンテキストを取得する、といったことも可能です。

筆者も `sitemcp`^{注6} というオープンソースプロジェクトを立ち上げていました。これは世の中の任意のサイトを自動的にMCPサーバに変換するツールです。指定したWebサイトをスクレイピングしてMarkdown形式に変換し、MCPとしてLLMに情報を渡すことを目的としていました。そのため公式がMCPサーバとしてドキュメントを提供するのを待たずに、同じような方法でドキュメントを得る手段として `sitemcp` は当時有用とされていました。

■ MCPの課題とドキュメント取得手段の変化

MCPは外部とのつながりを標準化したものとして爆発的に増えてきました。MCPの公式レジストリには何千ものサーバが登録されています。しかし、2025年後半になると、その限界やいくつかの課題が指摘され始めました。

まず第一に、MCPを大量に登録した場合にコンテキストウィンドウを圧迫するという問題があります。MCPはツールの説明文やパラメータの説明文が必須であるため、登録するMCPの数が増えてくると、その分だけコンテキストウィンドウが圧迫されてしまいます。これはMCPを呼び出すクライアント側の問題であり、MCPサーバやローカルMCPの問題ではない

注5) What is the Model Context Protocol (MCP)?
<https://modelcontextprotocol.io/>

注6) <https://github.com/ryoppippi/sitemcp>

Vibe Coding完全攻略

コードを1行も書かない、新時代のAIプログラミング手法

ことに注意が必要です^{注7}。

第二に、MCPは外部から情報を得るたびに何度も通信を行わなければなりません。必要なクエリなどを送って、必要最低限の情報が来るのを待つしか方法がなく、たとえば多くの情報を取得する際にはN+1問題のような非効率的な事象が発生します。ほかにも、現状MCPで取得した情報はLLMのコンテキストウィンドウで保持する以外の方法がなく、必要なときに必要な情報だけを取得して利用する、という手段が確立されていない点も欠点として挙げられます。

以上のことより、MCPは静的な情報を受け取る手段としては効率という点においてベストな方法ではないというのが、2025年12月の執筆時点での主流な考え方です。

ただ、FigmaやGitHubのような動的に情報に変化するサービスから情報を受け取る、という目的においてはMCPは現在でも有力な手法です。

MCPからCLI／ローカルドキュメントへの回帰

こういった流れを受けて、最近ではMCPによるドキュメントの受け渡しではなく、LLMが直接操作可能なCLIやMarkdownドキュメントをローカルに保存した状態を用意し、LLMが自発的に情報を取得できるしきみを整える方法に再び注目が集まりつつあります。

ローカルにダウンロードしたMarkdownであれば、grepなどのコマンドを使って検索や絞り込みを行うことができ、コンテキストを絞り込むことができます。たとえばJavaScriptランタイムであるBunは、bun-typesというパッケージにMarkdownドキュメントを直接同梱し、それをLLMに参照させることで、独自のAPIを使

えるように導く手段を提供しています^{注8}。また、WebフレームワークのHonoは、Hono CLI^{注9}を用意しCLI経由でドキュメントを取得するしきみを用意しています。TypeScript製のCLIライブラリ「gunshi」は、@gunshi/docsというパッケージを提供しており、node_modules内にサイトと同等のMarkdownをすべて同梱しているばかりか、自動的にClaude CodeやCursorのためのルールを設定するinit commandまで備えています^{注10}。

この流れは一見、RAGのようなコンセプトへの回帰に見えますが、そうではありません。コーディングエージェントが自発的にドキュメントを探索し、必要な情報を取得するという点でRAGと大きく異なります。ここ半年余りで、コーディングエージェントのCLIツールの使い方の精度は比べ物にならないほど向上しています。RAGでは適切なデータを持ってくるためにベクトルデータベースなどの前処理を必要としていましたが、現在では必要なドキュメントを渡せば、LLMが自発的に情報を取ってくれるのです。しかもMCPと違って外部と情報をやりとりすることなく、Web検索のように情報が見つかるかどうかを運任せにする必要もなく、効率および正確性においてかなり優れた手法だと言えるでしょう。2025年12月末現在では、この方法がベストだと筆者は結論付けています^{注11}。

仕様に沿っているかを 確認する



続いて、LLMの生成したコードが仕様に沿っているかを検証する方法について考えていきます。

注7) Fan et al., "MCPToolBench++: A Large Scale AI Agent Model Context Protocol MCP Tool Use Benchmark", arXiv:2508.07575, 2025
"Help or Hurdle? Rethinking Model Context Protocol-Augmented Large Language Models", arXiv:2508.12566, 2025

注8) <https://github.com/oven-sh/bun/blob/e9e93244cb3fee8bc4d734e7a4f3f2883eb1bf4a/packages/bun-types/scripts/build.ts>

注9) <https://github.com/honojs/cli>

注10) <https://gunshi.dev/guide/introduction/setup#llm-assisted-development>

注11) <https://ryoppiippi.com/blog/2025-12-14-publish-docs-on-npm-ja>

TODOリストによる実装過程のトラッキング

ここでは、LLMが仕様をどう扱うかについて述べていこうと思います。

最初に仕様が決められていて、その仕様に基づいた成果物になるようコードを書く、という手法は、読者のみなさんにも馴染み深いものだと思います。しかし、Vibe Codingにおいて最初にプロンプトで要件を与えるだけだと、途中で仕様を忘れたり、いつの間にか仕様とズレた実装になっていたりすることがあります。

仕様に沿った開発ができているかをトラッキングするためには、まずLLM用のTODOリストやプランを与えた仕様を基に構築させ、それに沿って実装させることが大事です^{注12}。たとえば、最近のAIツールでは、LLMがすべきことを忘れたり飛ばしたりしないように、まず仕様をチェックマーク付きのTODOリストとして落とし込み、それを1つずつ上から下になぞるように開発させることで、仕様に沿っているかどうかをトラッキングするしくみが一般化しています。このしくみを搭載しているツールとしては、Claude Codeやそれを補助するためのツール、GitHubの仕様駆動開発ツールSpec Kit^{注13}、Googleの開発するAIエディタAnti gravity^{注14}などがあります。

このような、やるべきことをLLMにリストアップさせ、それを順に実行させるという手法は、AIが仕様に沿った開発をしていることを保証する、1つのガードレールと言えると思います。これは実装中における補助の1つとなるでしょう。また、そのTODOリストをローカルに閉じておくだけではなく、GitHubのIssueやJiraのチケットなどと連動させるのも効果的な手段です。GitHubではghコマンド、Jiraで言えばJira MCPなどを使うことでそれが可

能になります。

Figma MCPによる仕様作成

FigmaのデザインをWebフロントエンドやモバイルアプリに落とし込むという開発手法は、現代ではとても一般的です。最近のVibe Codingでは、この流れと同じように、デザインツールを基に実装を進めることもよくあります。たとえばFigma MCPなどのツールを用いると、Figmaから必要な実装したい部分のスクリーンショットとHTMLのコードが取得できるため、LLMはそれをヒントにすることができます。視覚情報や特殊な形式の設計図のようなものを、LLMに効率的に渡すことも重要になってきます。

また、コーディングエージェントはCLIの使い方も飛躍的に向上しています。そのため、ドキュメントを与えるという目的以外にも、いろいろな情報を取得するためのCLIツールをLLMの手元に整備しておくことも重要です。たとえば、高速にファイルを検索できるfdやripgrepやのようなグレップツールを用意しておいたり、DockerコマンドやSQLコマンドなど、開発環境にアクセスできるツールもインストールしておいたりするとよいでしょう。

生成されたコードが正しいかを検証する



最後のステップとして、生成されたコードが正しいかを検証する方法を紹介します。これにはいくつか方法があります。全体を通して我々人類がエンジニアリングに対して向き合ってきた考え方に通ずるものがあります。

Lint/Formatter

現代の言語では、LintやFormatterが用意されていることが常識とされています。Lintとは、静的にコードに誤りがないかどうかをチェックするツールのことです。Formatterは、1行に何文字あるか、タブを使っているか、

注12) <https://platform.claude.com/docs/en/agent-sdk/todo-tracking>

注13) <https://github.com/github/spec-kit>

注14) <https://antigravity.google/>

Vibe Coding完全攻略

コードを1行も書かない、新時代のAIプログラミング手法

Quoteはシングルクオートなのかダブルクオートかなどなど、コードの形式を機械的にチェックまたは修正してくれるツールのことです。たとえばTypeScriptであれば、LinterとしてESLintやoxlint^{注15}、FormatterとしてPrettierやoxfmt^{注16}などがよく使われます。

また型安全な言語であれば、型チェッカーも同様の役割を果たします。LinterやFormatterと同じように、実行する前にバグやコードの誤りを見つけることができるという意味で有用です。これらの修正やチェックというのは決定的であるため、揺らぎの生じるLLMの出力を修正し、コードの質を担保するために非常に有効です。

ユニットテスト/E2Eテスト

ユニットテストを書くことは非常に重要です。Vibe Codingにおいても、TDD (テスト駆動開発)^{注17}に代表されるような「テストを最初に書いて、それを通す実装を行う」という流れを取ると、より品質の高い成果物が得られます。

テスト駆動開発におけるテストは、仕様書をコードに落としたものであり、コードが仕様に沿っているかを検証するための重要な手段です。ユニットテストはもちろんコードで表現されるため、静的な解析が可能であり、かつユニットテストというのは何度実行させても同じ結果が得られるものが保証されていることが必要です。そのため、ユニットテストを用意することで、LLMが生成したコードが実際に仕様に沿って実装されているか、バグなく正確に動くか、など動作の正確性を担保できるようになります。TypeScriptやJavaScriptであれば、VitestやJestなどがよく使われます。

2025年前半では、テストは人間が書き、実

装をコーディングエージェントに書かせる、という手法が最も品質が良いものとされてきました。ただ現在では、仕様に基づいてテストコードと実装を両方LLMに書かせても、品質の良いものができるという流れになっています。

それでもテストコードというのは静的解析の大事な手段の1つです。実装の途中であっても人間が定期的にテストの内容をチェックして、LLMにフィードバックを与えることはとても大切です。実装が終わったあとでも、実装中でもテストの内容がきっちり正しいかどうか、人間がきっちりコードリーディングをして判断すべきであるというのは言うまでもありません。

視覚的な検証手段

最新のLLMやコーディングエージェントは、画像などの視覚情報を扱うこともできます。たとえばPlaywright MCPやChrome DevToolsなどのMCPを使うと、実際にアプリケーションを動かして得たスクリーンショットなどの視覚情報を、LLMに直接渡すことができます。また、Playwrightテストなどで実際にスクリーンショットを見比べて、それをフィードバックするテストなども可能です。この組み合わせにより、コード上だけではわからない見た目の動作に関しても検証が可能になり、LLMへのフィードバックにつなげてコードの品質をさらに高めることにも役立ちます。

CI/CD/Git Hooksでの自動化

こうしたLintやテストなどは、ことあるごとに自動的に行うことで、LLMに自動でフィードバックを送ることができます。途中で変更を加えるたびにフィードバックをLLMに渡すことで、人間の介入を最小限に抑えながらも、コードの品質を高めていくことができます。フックするのは次のようなタイミングがよいでしょう。

- ・Gitにコードをコミットしたとき (CI/CDに組み込む)

注15) <https://oxc.rs/docs/guide/usage/linter>

注16) <https://oxc.rs/docs/guide/usage/formatter>

注17) ブログ記事「【翻訳】テスト駆動開発の定義」
<https://t-wada.hatenablog.jp/entry/canon-tdd-by-kent-beck>

- ・コーディングエージェントが作業を終えたとき (Claude Code Hooks など)



このように生成されたコードを決定的な手段で検証することによって、いわば守りの手段としてコードの品質を高めることができます。

コードレビュー

コードレビューを通じて、Vibe CodingでAgent Loopを回し、コードを改善していくことも重要です。CodeRabbit^{注18)}のようなコードレビュー特化型のエージェントを使用することで、人間がレビューする前に機械的に実行可能な指摘を洗い出せます。また、GitHub上でコードの行数を指定してコメントを付け、Claude CodeやCodexなどのエージェントに取得させ修正作業を行わせることで、人間が振り返り可能な形で修正作業を繰り返すことも可能です。

効果的な開発環境整備の例



最後に、これまで紹介してきた前処理、処理中、後処理におけるポイントをふまえて、簡単に筆者がフロントエンド開発で行っているコーディングの開発環境の例を紹介したいと思います。

ここではWebフロントエンド開発を例に挙げようと思います。

最初にライブラリをダウンロードしてきてきます。そのドキュメントも何らかの方法でなるべくローカルにダウンロードします。先ほど述べたとおり、BunやHonoなどのドキュメントをMarkdownで落とすことはもちろん、OSSのライブラリであれば、GitHubからコードベース自体をクローンしてきて、ローカルに置いておくこともあります。おすすめはリポジトリ管理ツールのghqを用いて自動的にクローンしてきて、リポジトリの場所を1カ所にまとめてお

くことです。

次に、LinterやFormatter、型チェッカーを設定します。ツールを選ぶ際は、できるだけ高速なツールを選びましょう。TypeScriptのバージョンはTypeScript 5を指定し、Linterにはoxlintやoxfmtを設定します。これらのLintなどのためのCI/CDのパイプラインの整備もちゃんとしておきます。

フロントエンドのデザインは、Figmaで作成したデザインをコードに起こします。そのために、MCPはFigma MCPとChrome DevToolsを導入します。これによってFigmaから直接データを受け取ることができ、なおかつコーディングエージェントはブラウザをコントロールできるようになるため、スクリーンショットを取れることはもとより、クリックや文字入力などによるインタラクティブなデバッグも可能です。

ここまで準備したら、仕様をLLMに伝えていきます。仕様をもとにやってほしいことをLLMと対話しながらTODOリストに記述させていき、PLAN.mdというファイルにまとめることが多いです。

あとは、コーディングエージェントはドキュメントを読み、テストを書き、実装を作り、ブラウザで動作を確認して、フォーマットのかかったコード、実際に動作するコードをGitHubにコミットしてくれます。もちろんGitHub上にCodeRabbitのようなコードレビューエージェントがいれば、ghコマンドでコメントを取得して、現在の開発に反映することもできます。

このようにして情報のインプット、コードの生成、レビューやLintの反映という3つの要素をぐるぐると回すことで、半自動的に品質の高いコードを生み出せる環境が整います。これをエージェントループと呼ぶことが多いですが、このようになるべく正確かつ多くの情報を与えてあげることがコード生成の質を上げることに繋がると考えています。

注18) <https://coderabbit.ai/>

Vibe Coding完全攻略

コードを1行も書かない、新時代のAIプログラミング手法

まとめ

従来であれば「このMCPが推奨される」といった議論が中心であったと想定されますが、現在のLLMやコーディングエージェントは十分に高性能であるため、LLMがいつでも自由に呼び出しできる状態でドキュメントを与え、自動的かつ決定的に検証できるしくみを整えることで、生成されるコードの質を高めることができます。この原則は、人間のエンジニアにも同様に当てはまります。適切な資料を用意して、適切な仕様を提供して、コードを書くたびにチェックを走らせる。従来から実践されてきた手法を着実に実行していくことが重要です。

もちろん、コーディングエージェントが理解しやすいようにLinterのエラーをわかりやすくする、CLIを呼び出しやすいものに設計する、MCPサーバやドキュメントサイトを最適化する

という課題もありますが、それらは開発者側の課題であるため本章では扱いません。

本章で議論をしてきた開発環境の重要性というのは、AIエージェントが現れる以前から、ソフトウェア工学の分野で多く議論されていたことに通ずるものがあります。クリーンアーキテクチャとは何か?どのようにテストを書いてテスト駆動開発をするのか?CIの重要性、ドキュメントを充実させる必要性、コードレビューはどのように行うべきか?などなど、人類が数十年をかけて向き合ってきた問題と不可分です。そのため、このように環境を整備することは、AIエージェントだけではなく人間にとっても有益なものになるでしょう。変化の激しい部分と、それまでの我々が人類として蓄えてきた知が交差して結実していると考えたととてもおもしろいものです。AIにも人間にも理想的な開発環境を整えていくことができれば、双方の生産性により大きく寄与することでしょう。SD

技術評論社



中井悦司 著
A5判/304ページ
定価(本体2,600円+税)
ISBN 978-4-297-14972-7

大好評
発売中!

仕事から学ぶ 生成AI入門

近年続々と発表される実用レベルの生成モデル論文を深く理解するため、本書は変分オートエンコーダ(VAE)、LSTMといった基礎モデルから、VQ-VAE、拡散モデル、Transformerといった最先端モデルの先駆けとなったモデルまで、その仕組みを数学的な詳細に偏らず、シンプルなサンプルコードと演習を通して解説します。Kerasを用いた実装を通して、各モデルの主要機能と生成モデルとしての動作原理を、実際に手を動かしながら理解することを目的としています。

こんな方におすすめ

- ・機械学習の基礎から生成AIを学びたい方
- ・生成AIのしくみをしっかりと基本から学びたい方